

# TP3

## Perceptrones

TP3 — Perceptrones  
Resultados experimentales sobre cuatro problemas

Sistemas de Inteligencia Artificial · ITBA · 1Q 2026

Bloque 1 / 4

# Validación: AND y XOR

---

# Qué pide el TP

---

Tres ejercicios + una validación previa, todo evaluado experimentalmente.

---

|      |                                                      |
|------|------------------------------------------------------|
| Ej 0 | Validación: AND con perceptrón escalón, XOR con MLP. |
|------|------------------------------------------------------|

---

|      |                                                         |
|------|---------------------------------------------------------|
| Ej 1 | Detección de fraude por <i>knowledge distillation</i> . |
|------|---------------------------------------------------------|

---

|      |                                                                    |
|------|--------------------------------------------------------------------|
| Ej 2 | Clasificación de dígitos manuscritos (10 clases, $28 \times 28$ ). |
|------|--------------------------------------------------------------------|

---

|      |                                                                      |
|------|----------------------------------------------------------------------|
| Ej 3 | Mismo problema, target: <b>accuracy</b> $\geq 98\%$ <b>en test</b> . |
|------|----------------------------------------------------------------------|

---

**Regla de evaluación:** `digits_test.csv` se evalúa *una sola vez*, al final, con el config ganador congelado. No se toca durante búsqueda de hiperparámetros.

## Ej 0 — AND con perceptrón escalón

### Setup

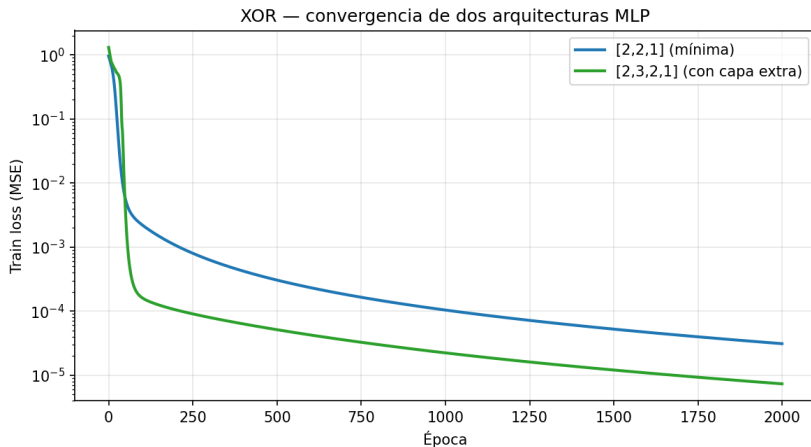
- Representación bipolar: entradas  $\{-1, +1\}$ , salida esperada  $[-1, -1, -1, +1]$ .
- Función de activación escalón (signo).
- Regla delta clásica:  $\Delta w = \eta (z - O) x$ .

| $x_1$ | $x_2$ | $z$ esperado |
|-------|-------|--------------|
| -1    | -1    | -1           |
| -1    | +1    | -1           |
| +1    | -1    | -1           |
| +1    | +1    | +1           |

**Resultado: error = 0 en pocas epochs. ✓**

Las cuatro muestras del AND son linealmente separables. La regla delta converge.

## Ej 0 — XOR con MLP: convergencia de dos arquitecturas



**El XOR no es separable linealmente.** Necesita capa oculta no lineal. Las dos arquitecturas convergen: **[2, 2, 1]** (mínima) y **[2, 3, 2, 1]** (con capa extra). Eje y

## Ej 0 — Qué nos dice el resultado del XOR

**Resultado clásico** (Minsky & Papert, 1969 → Rumelhart & Hinton, 1986):

*El perceptrón simple no resuelve XOR; un MLP con una capa oculta no lineal sí.*

**Lo que la convergencia confirma:**

- **Forward pass** de la red está bien implementado.
- **Backpropagation** computa los gradientes correctos (si no, el loss no bajaría).
- Las activaciones **tanh** y sus derivadas están bien definidas.
- El loss **MSE** y su gradiente también.

Esta validación es la base para los ejercicios siguientes: la misma pipeline se reutiliza para clasificación de dígitos cambiando solo activación final (softmax) y loss (cross-entropy).

### Validación completa.

El AND lo resolvió un perceptrón simple con error cero.  
El XOR lo resolvieron dos arquitecturas distintas de MLP.

Siguiente: **detección de fraude**  
con un perceptrón *simple* (una sola capa).

Bloque 2 / 4

# Ej 1 — Knowledge Distillation

---



## Ej 1 — El problema

---

**Knowledge distillation:** replicar la salida del BigModel pre-entrenado con un TinyModel (perceptrón simple).

**Datos:** `fraud_dataset.csv`

9 features (amount, session, account\_age, ...)

**Target de entrenamiento:**

`big_model_fraud_probability` (continuo en  $[0, 1]$ ).

**Ground truth de evaluación:**

`flagged_fraud` (binaria 0/1).

**Regla del enunciado:**

`flagged_fraud` *nunca* se usa para entrenar. Solo para métricas operativas (precision, recall, F1) post-hoc.

**Salida esperada en  $[0, 1]$   $\Rightarrow$**   
activación sigmoide.

## Ej 1 — Lineal vs No-Lineal: MSE de distillation

Mismo dataset, K-fold = 5 estratificado, mismas features. Diferencia: presencia de la sigmoide.

| Modelo               | MSE test (mean $\pm$ std)   |
|----------------------|-----------------------------|
| Lineal (Adaline)     | 0,0266 $\pm$ 0,0008         |
| No-Lineal (sigmoide) | 0,0110 $\pm$ 0,0004 (−59 %) |

### ¿Por qué la sigmoide gana tan claro?

El target es una probabilidad acotada en  $[0, 1]$ . La sigmoide *naturalmente* mapea su salida a ese rango. El lineal puede producir valores fuera de  $[0, 1]$  y el MSE penaliza fuerte esa salida fuera de dominio.

## Ej 1 — El threshold default es engañoso

**Modelo no-lineal con threshold** = 0,5 (default obvio para sigmoide):

| Precision | Recall | F1    | Accuracy |
|-----------|--------|-------|----------|
| 0.333     | 1.000  | 0.499 | 0.768    |

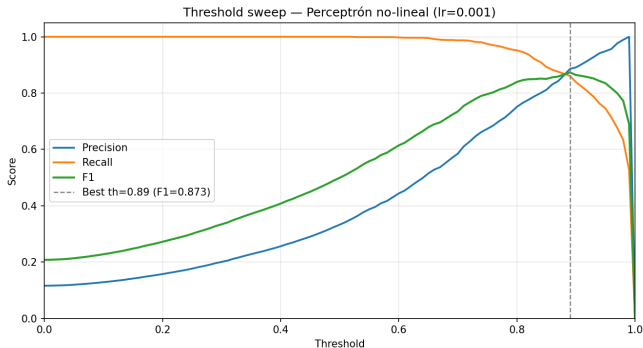
Esto luce como un modelo terrible. **No es el modelo, es el threshold.**

**Las salidas sigmoide se concentran en la parte alta:**

- Mediana de los scores de fraude: **0.99**.
- No-fraudes *no* bajan a 0.05; bajan a 0.4–0.6.

El modelo aprendió un buen **ranking**, no una calibración perfecta. Threshold bajo  
⇒ sobre-detección.

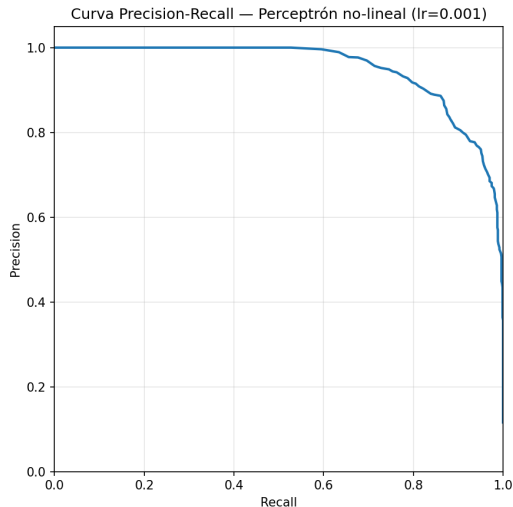
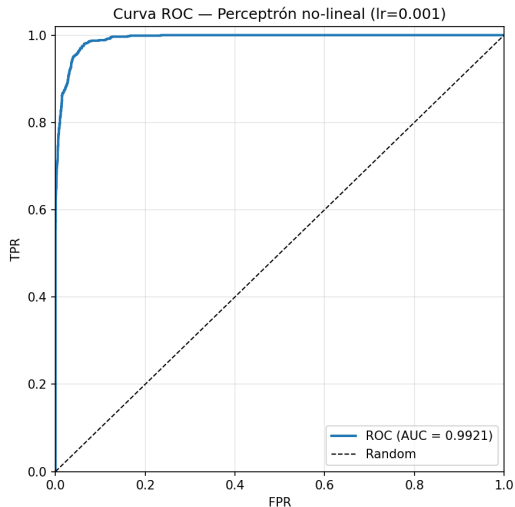
## Ej 1 — Threshold sweep



**Óptimo por F1: threshold = 0.89.**

Precision 0,886, recall 0,861, F1 0,873, accuracy 0,971.

# Ej 1 — Curvas ROC y Precision-Recall



## Ej 1 — Recomendación de umbral

La elección depende del costo asimétrico falso positivo / falso negativo.

| Threshold               | Precision    | Recall       | F1           | FP        | Caso de uso             |
|-------------------------|--------------|--------------|--------------|-----------|-------------------------|
| 0.50 (default)          | 0.333        | 1.000        | 0.499        | 1743      | —                       |
| <b>0.89</b> (óptimo F1) | <b>0.886</b> | <b>0.861</b> | <b>0.873</b> | <b>96</b> | auditoría manual barata |
| 0.95                    | 0.949        | 0.746        | 0.835        | 35        | FP costosos             |
| 0.99                    | 1.000        | 0.527        | 0.690        | 0         | cero tolerancia a FP    |

El modelo entrega scores. La política de threshold es decisión de negocio:

$\text{eftmargin}=1.4\text{em}$ Auditoría manual barata  $\Rightarrow$  threshold

$\approx 0,89$ .  $\text{eftmargin}=1.4\text{em}$ Falsos positivos costosos  $\Rightarrow$  threshold  $\geq 0,95$ .

Bloque 3 / 4

# Ej 2 — Clasificación de dígitos

---

## Ej 2 — El problema

---

### Clasificación multiclase de dígitos manuscritos (0–9).

#### Datos:

- Train: `digits.csv` (12.449 muestras)
- Test: `digits_test.csv` (2.498)

Cada muestra: vector de 784 floats  $\in [0, 1]$   
( $28 \times 28$  flatten).

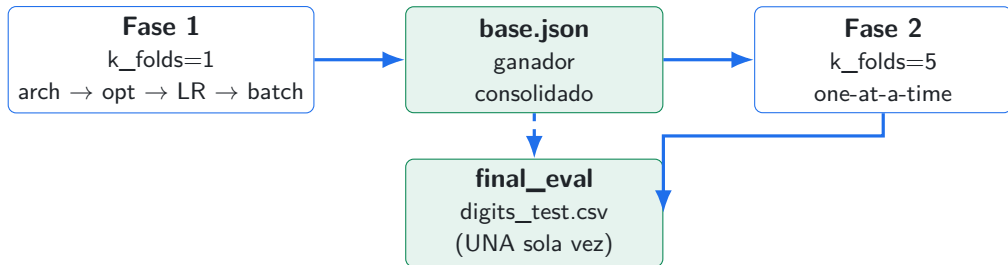
#### Arquitectura final:

- Input: 784
- Hidden 1: 100 (ReLU)
- Hidden 2: 50 (ReLU)
- Output: 10 (Softmax)

**Regla:** el test set se evalúa una sola vez al final.  
Búsqueda de hiperparámetros sólo sobre `digits.csv`.



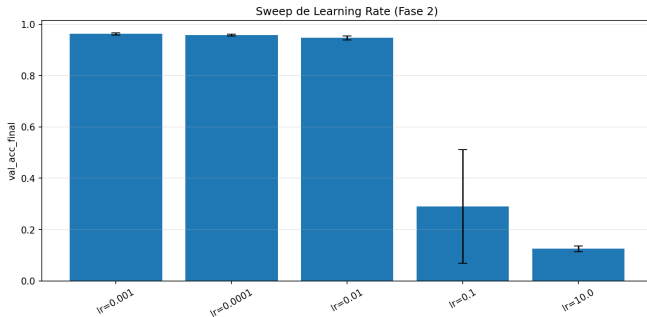
## Ej 2 — Workflow disciplinado: dos fases



**Fase 1:** sweeps secuenciales para reducir el espacio de búsqueda.

**Fase 2:** K-fold=5 para validar la elección con comparativas robustas.

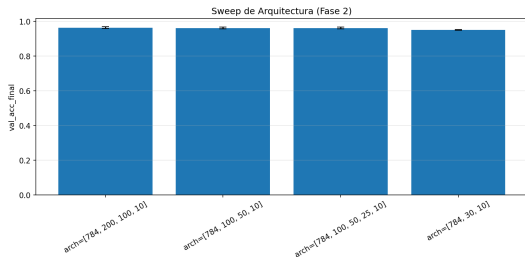
## Ej 2 — Sweep de Learning Rate (Fase 2)



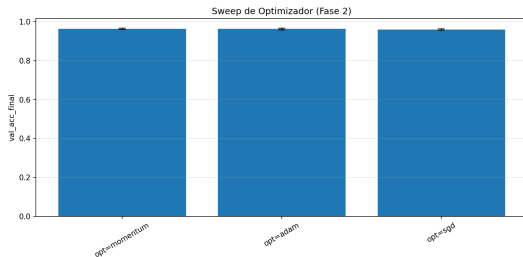
**Extremos catastróficos:**  $lr = 10$  deja la red en accuracy random (12 %);  $lr = 0,1$  también diverge.

**Óptimo:**  $lr = 0.001$  (consistente con la elección de Fase 1).

## Ej 2 — Sweeps de Arquitectura y Optimizador



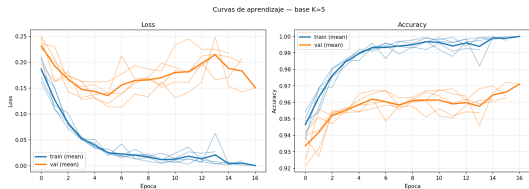
**wider** (200/100) gana en val por 0.17pp  
pero tarda  $3.6\times$  más → empate técnico



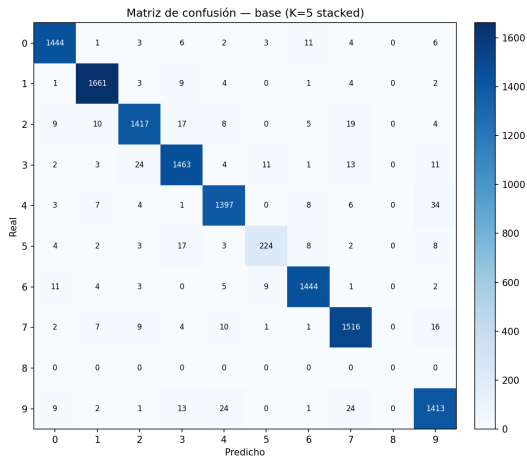
Adam y Momentum empatan dentro del std.  
SGD claramente atrás.

**Fase 2 valida más que descubre.** Solo el sweep de LR muestra rangos catastróficos.  
Arch, opt e init quedan en empate técnico.  
⇒ La metodología coarse-to-fine de Fase 1 ya hizo el grueso del trabajo.

## Ej 2 — Curvas y matriz del modelo base (val K=5)



Convergencia limpia ~ 5 epochs.  
Sin overfitting visible.



La clase 8 colapsa: precision = recall = 0.

La red no la predice nunca.

val K-fold=5: **96.22 %**



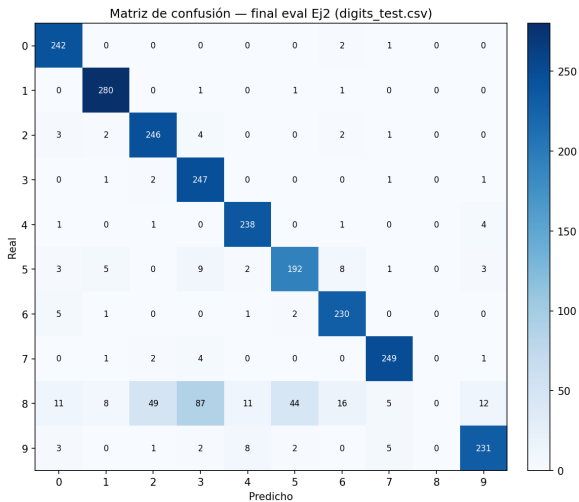
test (digits\_test.csv): **86.30 %**

**Drop de 10 puntos porcentuales.**

**Interpretación: distribution shift, no underfitting.**

La matriz de confusión sobre test (siguiente slide) confirma: las confusiones se reparten por todo el grid, no es una clase específica

## Ej 2 — Matriz de confusión sobre digits\_test.csv



Bloque 4 / 4

# Ej 3 — Pack C y conclusiones

---

## Ej 3 — Hipótesis y plan

---

**Hipótesis:** el techo del Ej 2 es por distribution shift, no por falta de capacidad.

**Plan secuencial:**

1. **Más datos.** Concatenar `more_digits.csv` (15.742 muestras adicionales). Si llega a  $\geq 0,98$ , listo.
2. **Pack C.** Activar regularización (L2, dropout, augmentation gaussiana, lr scheduling) y combinaciones.
3. **Capacidad.** Si nada llega, probar `arch_wider`.

**Cero cambios al modelo en el paso 1.** Solo cambia un campo del JSON: `extra_csv_paths`.



### Ej 3 — El movimiento dominó

---

val  $K=5$ : 96,22 %  $\rightarrow$  97,06 %

test: 86,30 %  $\rightarrow$  96,36 %

**+10 puntos porcentuales en test.**  
Cero cambios al modelo. Solo más datos.

**Macro F1:** 0,857  $\rightarrow$  0,959.

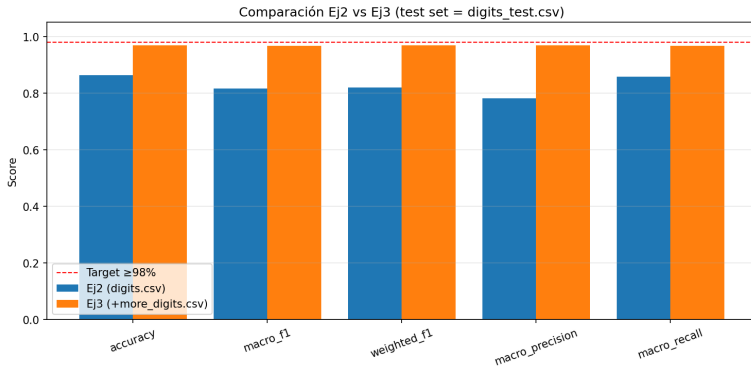
La clase 8 que estaba colapsada en Ej 2 ahora se predice correctamente. Más datos cubrieron el agujero de cobertura. Confirma la hipótesis del shift.

## Ej 3 — Tabla de resultados Pack C

| Config                                         | val K=5       | test          | $\Delta$ vs base_extra |
|------------------------------------------------|---------------|---------------|------------------------|
| base_extra_data                                | 0.9706        | 0.9636        | — (referencia)         |
| + L2 (1e-4)                                    | 0.9758        | 0.9656        | +0,20 pp               |
| + dropout (p=0.2)                              | 0.9750        | 0.9628        | −0,08 pp               |
| + L2 + dropout                                 | 0.9772        | 0.9604        | −0,32 pp               |
| <b>+ L2 + aug (<math>\sigma = 0,05</math>)</b> | <b>0.9760</b> | <b>0.9688</b> | <b>+0,52 pp</b>        |
| + L2 + aug + dropout                           | 0.9768        | 0.9656        | +0,20 pp               |
| + wider + L2 + aug                             | 0.9777        | 0.9640        | +0,04 pp               |

**Ganador:** L2 + augmentation gaussiana ( $\sigma = 0,05$ ) → **96.88 % en test.**

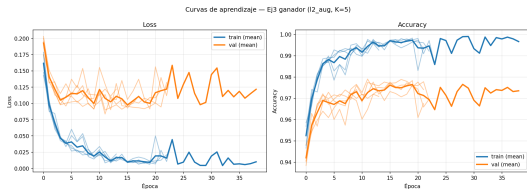
## Ej 3 — Comparación visual con Ej 2



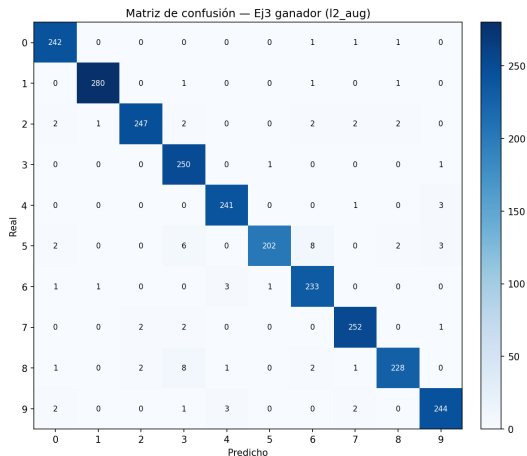
Línea roja: target  $\geq 98\%$ .

**El salto principal viene del baseline con extra data, no de la regularización fina.**

## Ej 3 — Modelo ganador (l2\_aug)



Convergencia más lenta que Ej 2 base (best\_ep ~ 10 vs 5) por la regularización.



Matriz sobre digits\_test.csv: clase 8 ya predice correctamente.

## Ej 3 — Análisis: qué movió la aguja, qué no

### Lo que ayudó:

- **Más datos: +10 pp.** El movimiento dominante.
- **L2: +0,20 pp** consistente.
- **Aug gaussiana: +0,32 pp** *adicional* sobre L2 (sinergia).

### Lo que no ayudó:

- **Dropout:** gana en val (+0,44pp), *pierde* en test (−0,08pp). Reduce varianza al fold, no al shift.
- **wider [784, 200, 100, 10]:** mejor en val, peor en test. Más capacidad memoriza el train. **Descarta que falte capacidad.**
- **Combinar todo:** L2+dropout+aug peor que L2+aug solo. La regularización compite consigo misma cuando se exagera.

## Ej 3 — Por qué no llegamos al 98 %

**Mejor: 96.88 %. Faltan 1.12 pp.**

**Tres hipótesis ordenadas por probabilidad:**

**1. Augmentation insuficiente.**

Gaussian noise es *isotrópico* (independiente por píxel). El shift parece ser de *estilo de escritura*: rotación pequeña, traslación, grosor de trazo. Pide augmentation **geométrica**, no por noise. No la implementamos en este TP.

**2. Sub-representación en val interno del final\_eval.**

Val 10 % random para early stopping. Si los digits “difíciles” del test no están ahí, la red para antes de tiempo.

**3. Capacidad:** improbable, *wider* sobreajustó.

**Próximo paso:** augmentation por rotación/traslación pequeñas; reconsiderar arquitectura solo después.

# Conclusiones del TP

## Tres ideas para llevarse:

1. **Una buena pipeline gana más que tunear hiperparámetros.**  
El +10pp del Ej 3 vino de *datos*, no de modelo.
2. **Los empates técnicos son más comunes que los wins claros.**  
Hay que elegir por parsimonia, no por 0.05pp en val.
3. **“Mejor en val”  $\neq$  “mejor en test” cuando hay shift.**  
Dropout fue el ejemplo de manual: ganador en val, perdedor en test.

| Item                                                       | Status             |
|------------------------------------------------------------|--------------------|
| Ej 0 — step AND, MLP XOR                                   | ✓                  |
| Ej 1 — Knowledge distillation + threshold sweep            | ✓                  |
| Ej 2 — sweeps + final_eval + análisis del shift            | ✓                  |
| Ej 3 — <a href="#">more_digits.csv</a> + Pack C + análisis | ✓                  |
| <b>Ej 3 — target <math>\geq</math> 98 %</b>                | <b>✗ (96.88 %)</b> |

# Gracias.

Preguntas.

---